



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

**NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA**



**ACCREDITED GRADE 'A'
BY NAAC**



**PERFECT SCORE OF 150/150 AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS**

1



Applied Machine Learning

Unit 07 – Lecture 22: Hierarchical Clustering

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Merge the two closest clusters. Repeat.

Topic focus: **what “closest” means for two sets**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.; Ester, Kriegel, Sander and Xu, “A density-based algorithm for discovering clusters (DBSCAN),” KDD 1996.

Quick Links

A tree, not a partition

The algorithm

Single linkage by hand

Complete linkage by hand

Five linkages

Lance-Williams

Shape and chaining

Inversions

Cutting the tree

Time and space

Divisive

Summary

Agenda

A tree, not a partition

The algorithm and the six points

Single linkage, by hand

Complete linkage, on the same points

The distance between two clusters

One formula for all five

What linkage does to cluster shape

Centroid linkage can go backwards

Cutting the dendrogram

Time and space requirements

Agglomerative against divisive

Summary

How this lecture works

Every idea appears twice

1. **By hand** — six points, a 6×6 table, and a pen. Small enough that every number can be checked.
2. **In code** — the same numbers out of `scipy.cluster.hierarchy`, agreeing to twelve decimal places, so you can see the library is doing exactly this and nothing magic.

Commit before you look

Six slides ask you to predict something before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

The centrepiece is a **complete agglomerative run on six points, worked twice** — once under single linkage and once under complete linkage, with the proximity matrix rewritten after every merge. That arithmetic is the most examinable thing in the lecture.

One seed for the lecture, four that fix the data

The randomness

HAC is deterministic — same points, same tree. The only arbitrary seed is **0**: the point sets used for timing, the PCA solver, one KMeans, one BisectingKMeans.

The data, and one sweep

Blobs, moons and bands at `random_state=0`; the bridge set at `seed=3`. These *define which points exist*. The inversion experiment draws a new set every trial — that is the measurement — off one stream.

The one thing a seed cannot fix

Every number in this deck reproduces exactly *except the seconds*. Timings depend on your processor and BLAS build. Quote the operation count, never the clock.

Where we are

The last two lectures

What clustering is for, and then a whole lecture on one question: *what does it mean for two objects to be close?*

The output of all that

A **proximity matrix** — one number for every pair. That table is the input to today's algorithm, and today's algorithm needs nothing else.

Today's one question

I can measure the distance between two *points*. What is the distance between two *sets* of points?

And the answer is: pick one

Five standard answers, all reasonable. On the same six points they build **different trees**; on the same 300 points they score anywhere from ARI 0.00 to ARI 1.00.

Partitioning asks for k . Hierarchy refuses.

Partitioning (next lecture)

You supply k . You get *one* partition into k groups. Change your mind about k and you run it again.

Hierarchical (today)

You supply nothing. You get *all of them at once*:

$$\mathcal{P}_n, \mathcal{P}_{n-1}, \dots, \mathcal{P}_2, \mathcal{P}_1$$

one partition for every k , and each one refines the next.

Nested means drawable

Every cluster of $\mathcal{P}_k$ sits inside a cluster of $\mathcal{P}_{k-1}$: two clusters at any two levels are disjoint, or one contains the other. Never partially overlapping — which is exactly the condition for the family to be drawn as one tree, a **dendrogram**.

Choose k *after* seeing the tree, not before running the algorithm.

Reading a dendrogram — and one common misreading

What is in the picture

- **Leaves** = the n objects.
- **Internal nodes** = clusters; $n - 1$ of them, one per merge.
- **Height** = the distance between the two clusters when they merged. *The only quantitative information here.*
- **Horizontal cut** at h : the components below it are the clusters.

What is not in the picture

The **left-to-right order of the leaves means nothing**. Any subtree may be flipped; there are 2^{n-1} drawings of one tree. Two adjacent leaves need not be similar — to judge that, follow both up to their first common ancestor and read *its* height.

Two directions

Agglomerative (AGNES): bottom-up. Today.
Divisive (DIANA): top-down. Last section.

Hierarchical agglomerative clustering, in nine lines

Input : proximity matrix D over n objects, a linkage rule L

Output: a sequence of $n-1$ merges with their heights

$C = \{ \{1\}, \{2\}, \dots, \{n\} \}$ # n singleton clusters

for step in $1 \dots n-1$:

$(A, B) = \text{argmin over distinct pairs in } C \text{ of } L(A, B)$

 record the merge (A, B) at height $L(A, B)$

$C = C - \{A\} - \{B\} + \{A \text{ union } B\}$ # replace the pair by the union

 update the row/column of the new cluster in D

return the recorded merges

Everything is hidden in L

L is the rule for the distance between two *sets*.

Change L , change the tree.

Greedy and irrevocable

No reassignment, no convergence test, no random start. Deterministic — and **nothing is ever undone**.

The input is the matrix, not the data

What HAC needs

The $n \times n$ table D with $d_{ij} = 0$ and $d_{ij} = d_{ji}$. Only the $\binom{n}{2}$ entries above the diagonal are used.

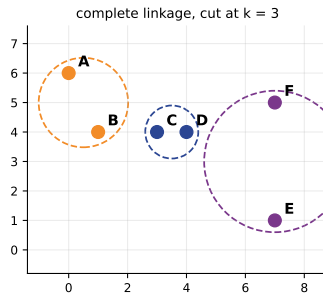
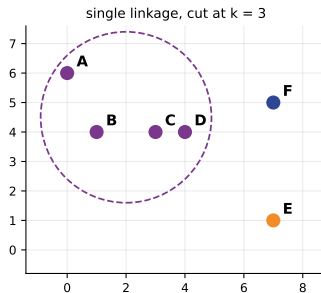
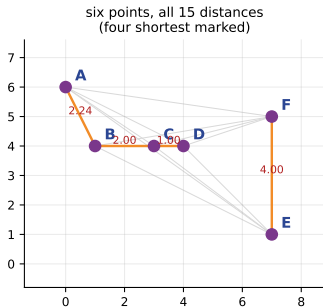
The triangle inequality is **not** required. A Euclidean embedding is **not** required — except for centroid and Ward.

Why that matters

A table of edit distances between DNA sequences. A matrix of “how often did these two customers buy the same thing”. HAC runs on both.

k -means cannot: it has to *average* points, and there is no average of two DNA sequences.

Six points — everything today is worked on these



$A(0,6)$ $B(1,4)$ $C(3,4)$ $D(4,4)$ $E(7,1)$ $F(7,5)$

Chosen so that all **fifteen** pairwise distances are distinct — no ties to argue about.

By hand: the initial proximity matrix

	A	B	C	D	E	F
A	--	2.2361	3.6056	4.4721	8.6023	7.0711
B	2.2361	--	2.0000	3.0000	6.7082	6.0828
C	3.6056	2.0000	--	1.0000	5.0000	4.1231
D	4.4721	3.0000	1.0000	--	4.2426	3.1623
E	8.6023	6.7082	5.0000	4.2426	--	4.0000
F	7.0711	6.0828	4.1231	3.1623	4.0000	--

Every entry is $\sqrt{\text{small integer}}$

$\sqrt{1}, \sqrt{4}, \sqrt{5}, \sqrt{9}, \sqrt{10}, \sqrt{13}, \sqrt{16}, \sqrt{17}, \sqrt{18}, \sqrt{20}, \sqrt{25}, \sqrt{37}, \sqrt{45}, \sqrt{50}, \sqrt{74}$

— integer Δx and Δy , so every distance is checkable in your head: $d(A, E) = \sqrt{7^2 + 5^2} = \sqrt{74} = 8.6023$.

Two numbers to remember

Smallest: $d(C, D) = 1.0000$.

Largest: $d(A, E) = 8.6023$.

Both will turn up as merge heights before the lecture is over.

Single linkage: nearest neighbour

$$d_{\text{single}}(A, B) = \min_{a \in A, b \in B} d(a, b)$$

The update rule

$$d(A \cup B, K) = \min(d(A, K), d(B, K))$$

The new row of the matrix is the **element-wise minimum** of the two old rows.

No arithmetic at all

Nothing is recomputed from the raw points. You look at two numbers and keep the smaller one. This is why six points take four minutes with a pen — and why you will be asked to do it.

Predict first #1 — which pair merges first?

Commit to an answer

Six singleton clusters. Look at the matrix on the previous slide.

Which pair merges first, and does the linkage rule matter?

Predict first #1 — which pair merges first?

Commit to an answer

Six singleton clusters. Look at the matrix on the previous slide.

Which pair merges first, and does the linkage rule matter?

$\{C\} + \{D\}$ at height **1.0000** — and **no**, the rule does not matter.

With all clusters singletons, every linkage rule reduces to the same thing: min, max and mean over a set of *one* cross distance are all that one distance. **The first merge is always the smallest entry of the original matrix, whichever linkage you chose.** The linkages only begin to differ at step 2.

By hand, step 1: merge C and D

New row = element-wise **minimum** of the C and D rows:

$\min(3.6056, 4.4721) = 3.6056$ against A ; $\min(2, 3) = 2$ against B ;
 $\min(5.0000, 4.2426) = 4.2426$ against E ; $\min(4.1231, 3.1623) = 3.1623$ against F .

	A	B	E	F	CD	
A	--	2.2361	8.6023	7.0711	3.6056	
B	2.2361	--	6.7082	6.0828	2.0000	<-- smallest
E	8.6023	6.7082	--	4.0000	4.2426	
F	7.0711	6.0828	4.0000	--	3.1623	
CD	3.6056	2.0000	4.2426	3.1623	--	

Five clusters left. The matrix has shrunk by one row and one column.

Predict first #2 — where does B go?

Commit before the matrix is updated

B is 2.2361 from A . B is 2.0000 from C and 3.0000 from D .

Does B pair up with A , or join $\{C, D\}$? And what if the rule were max instead of min?

Predict first #2 — where does B go?

Commit before the matrix is updated

B is 2.2361 from A . B is 2.0000 from C and 3.0000 from D .

Does B pair up with A , or join $\{C, D\}$? And what if the rule were max instead of min?

Under min: $d(B, CD) = \min(2, 3) = \mathbf{2.0000} < 2.2361$, so B **joins** $\{C, D\}$.

Under max: $d(B, CD) = \max(2, 3) = \mathbf{3.0000} > 2.2361$, so B **pairs with** A .

This one number decides the entire rest of the run. Two linkages, two trees, from the choice between 2.0000 and 3.0000 on the same two original distances.

By hand, steps 2 and 3

Step 2: merge *B* with *CD* at 2.0000

	BCD	A	E	F
A	2.2361	--	8.6023	7.0711
E	4.2426	8.6023	--	4.0000
F	3.1623	7.0711	4.0000	--
BCD	--	2.2361	4.2426	3.1623

Step 3: merge *A* with *BCD* at 2.2361

	E	F	ABCD
E	--	4.0000	4.2426
F	4.0000	--	3.1623
ABCD	4.2426	3.1623	--

Notice what is happening

Step 1 attached a point. Step 2 attached a point. Step 3 attached a point. One cluster is eating the data one object at a time — and the next slide shows it keep going.

By hand, steps 4 and 5 — and the answer

Step 4: F joins at 3.1623

	E	FABCD
E	--	4.0000
FABCD	4.0000	--

$d(F, ABCD) = 3.1623 < d(E, F) = 4.0000$, so F abandons E .

Step 5: E joins at 4.0000. Done.

Merge heights, in order

1.0000, 2.0000, 2.2361, 3.1623, 4.0000

All five merges attached a single point to one growing cluster

That behaviour has a name: **chaining**. Single linkage grows a cluster by absorbing whatever is nearest to *anything* already inside, so clusters extend along paths of close neighbours. Hold that thought — it is both the method's superpower and its catastrophe.

Complete linkage: farthest neighbour

$$d_{\text{complete}}(A, B) = \max_{a \in A, b \in B} d(a, b)$$

$$d(A \cup B, K) = \max(d(A, K), d(B, K))$$

What it insists on

Merging is only cheap if every member of one cluster is close to every member of the other. One distant pair vetoes the merge.

The height has a meaning

The height at which a cluster is formed is exactly its **diameter** — the largest distance between any two of its members. That is a genuinely interpretable number, which single linkage heights are not.

By hand, complete linkage: steps 1 and 2

Step 1: $C+D$ at 1.0000; new row = max

	F	A	B	E
F		CD		
A		--	2.2361	8.6023
	7.0711	4.4721		
B		2.2361	--	6.7082
	6.0828	3.0000		
E		8.6023	6.7082	--
	4.0000	5.0000		
F		7.0711	6.0828	4.0000
	--	4.1231		
CD		4.4721	3.0000	5.0000
	4.1231	--		

Step 2: $A+B$ at 2.2361

	E	F	CD
E			
	AB		
	--	4.0000	5.0000
F	8.6023	--	4.1231
	4.0000		
CD	7.0711	5.0000	4.1231
	5.0000		--
AB	4.4721	8.6023	7.0711
	8.6023	4.4721	4.4721
	--		

Compare the step-1 matrix with the single-linkage one: four entries changed and every one of them **grew**. $d(B, CD)$ went from 2.0000 to 3.0000, and the run turned.

By hand, complete linkage: steps 3, 4, 5

Step 3: $E+F$ at 4.0000

	CD	AB	EF
CD	--	4.4721	5.0000
AB	4.4721	--	8.6023
EF	5.0000	8.6023	--

Step 4: $CD+AB$ at 4.4721

	EF	CDAB
EF	--	8.6023
CDAB	8.6023	--

Step 5, and the heights

Merge at $8.6023 = d(A, E)$ — the largest distance in the data, which is the diameter of the whole set.

1.0000, 2.2361, 4.0000, 4.4721, 8.6023

Check the intermediate numbers

$4.4721 = d(A, D)$, the *largest* of the four cross distances between $\{A, B\}$ and $\{C, D\}$: $\{2, 3, 3.6056, 4.4721\}$. Complete linkage always reports the worst pair.

The two runs, side by side

step	single-linkage merge	height	complete-linkage merge	height
1	{C} + {D}	1.0000	{C} + {D}	1.0000
2	{B} + {CD}	2.0000	{A} + {B}	2.2361
3	{A} + {BCD}	2.2361	{E} + {F}	4.0000
4	{F} + {ABCD}	3.1623	{CD} + {AB}	4.4721
5	{E} + {FABCD}	4.0000	{EF} + {CDAB}	8.6023

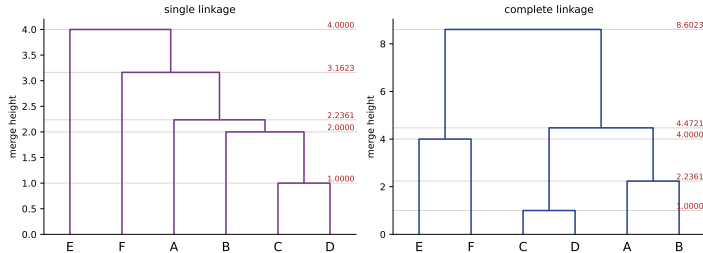
They agree only at step 1

After that, nothing in common: not the pairs, not the heights, not the shape.

Complete heights $\geq$ single heights

A maximum over a set cannot be below its minimum. So a complete-linkage tree is stretched vertically. **Never compare a height from one linkage with a height from another.**

Drawn: the two dendrograms



Left, single linkage: a **staircase**, one point per step. Right, complete linkage: three pairs, then two groups, then the root. Same six points. Note the vertical scales differ by a factor of two — and note that single linkage draws *E* and *F* adjacent although they are the *last* two things joined.

The flat clusterings, and the code behind them

```
for k in (2, 3, 4):
    print(k, fcluster(linkage(P6, "single"), k, "maxclust"),
          fcluster(linkage(P6, "complete"), k, "maxclust"))
```

k = 2	single	: {ABCDF} {E}	complete: {EF} {ABCD}
k = 3	single	: {ABCD} {F} {E}	complete: {EF} {CD} {AB}
k = 4	single	: {BCD} {A} {F} {E}	complete: {E} {F} {CD} {AB}

And the hand run is exact

```
merge heights in order: 1.0000, 2.0000, 2.2361, 3.1623, 4.0000
scipy heights           : 1.0000, 2.0000, 2.2361, 3.1623, 4.0000
match to 1e-12: True
```

One warning before we generalise: ties

These six points were chosen to have no ties

All fifteen distances differ. Real data — integer grids, duplicated rows, rounded measurements — has ties all the time.

What happens then

The merge order becomes **implementation-dependent**. SciPy's documentation says so in as many words. Two correct programs can return two different trees from the same matrix.

What to do

In an examination: say which tie-break you used and carry on.

In the lab: if the tree changes when you change library version, check for ties *before* assuming you have a bug.

Five answers to the one question

With n_A , n_B the cluster sizes and c_A , c_B the centroids:

$$d_{\text{single}}(A, B) = \min_{a,b} d(a, b) \quad \text{nearest neighbour}$$

$$d_{\text{complete}}(A, B) = \max_{a,b} d(a, b) \quad \text{farthest neighbour}$$

$$d_{\text{average}}(A, B) = \frac{1}{n_A n_B} \sum_a \sum_b d(a, b) \quad \text{UPGMA}$$

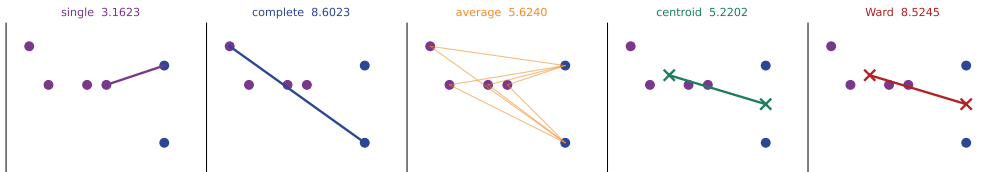
$$d_{\text{centroid}}(A, B) = \|c_A - c_B\|_2 \quad \text{UPGMC}$$

$$d_{\text{ward}}(A, B) = \sqrt{\frac{2n_A n_B}{n_A + n_B}} \|c_A - c_B\|_2 \quad \text{minimum variance}$$

First three: any dissimilarity — they only look up entries of D , so edit distances, cosine and Jaccard are all fine.

Last two: Euclidean only — a centroid needs a vector space, and SciPy will not stop you handing ward a cosine matrix.

Drawn: what each rule actually measures



The same two clusters, $\{A, B, C, D\}$ against $\{E, F\}$, measured five ways. Single takes one of the eight cross distances; complete takes another; average takes all eight; centroid and Ward work from the crosses.

By hand: all five, for $\{A, B, C, D\}$ against $\{E, F\}$

Eight cross distances, from the original matrix:
 $\{8.6023, 7.0711, 6.7082, 6.0828, 5.0000, 4.1231, 4.2426, 3.1623\}$

```
single   min           = 3.1623           (the pair D,F)
complete max         = 8.6023           (the pair A,E)
average  mean         = 44.9924 / 8 = 5.6240
centroid ||cA - cB|| = 5.2202   cA = (2.0000, 4.5000)   cB = (7.0000, 3.0000)
ward     sqrt(2*nA*nB/(nA+nB))*||cA-cB|| = sqrt(2.6667)*5.2202 = 8.5245
```

The centroids, by hand

$$c_A = \frac{1}{4}(0+1+3+4, 6+4+4+4) = (2.0, 4.5)$$

$$c_B = \frac{1}{2}(7+7, 1+5) = (7.0, 3.0)$$

$$\|c_A - c_B\| = \sqrt{5^2 + 1.5^2} = \sqrt{27.25} = 5.2202$$

Five numbers between 3.16 and 8.60, all correctly describing the distance between the *same* two clusters.

Where Ward's formula comes from

The only one of the five with an objective

Within-cluster sum of squares $\text{ESS}(A) = \sum_{a \in A} \|a - c_A\|^2$: zero at the singleton level, maximal at the root. **Ward merges the pair that increases the total by the least.**

$$\Delta(A, B) = \text{ESS}(A \cup B) - \text{ESS}(A) - \text{ESS}(B) = \frac{n_A n_B}{n_A + n_B} \|c_A - c_B\|^2$$

SciPy reports $\sqrt{2\Delta}$

$\Delta = \frac{4 \cdot 2}{6} \times 27.25 = 36.3333$, and
 $\sqrt{2 \times 36.3333} = \mathbf{8.5245}$ — the number in the block above. The factor of 2 makes the height equal $d(a, b)$ for two singletons.

One sentence to remember

Ward is k -means' objective run greedily bottom-up: compact, similarly sized, roughly spherical clusters. The sensible default on numeric data.

The Lance-Williams recurrence (1967)

$$d(A \cup B, K) = \alpha_A d(A, K) + \alpha_B d(B, K) + \beta d(A, B) + \gamma |d(A, K) - d(B, K)|$$

method	α_A	α_B	β	γ
single	1/2	1/2	0	-1/2
complete	1/2	1/2	0	+1/2
average	$n_A/(n_A+n_B)$	$n_B/(n_A+n_B)$	0	0
centroid	$n_A/(n_A+n_B)$	$n_B/(n_A+n_B)$	$-n_A n_B / (n_A + n_B)^2$	0
Ward	$(n_A + n_K) / T$	$(n_B + n_K) / T$	$-n_K / T$	0

$T = n_A + n_B + n_K$. **Centroid and Ward act on squared distances**; take the square root at the end.

The new distance depends only on **three numbers already in the matrix**. The raw points are never revisited.

By hand: min and max fall out of the formula

Put $\alpha_A = \alpha_B = 1/2$, $\beta = 0$, $\gamma = -1/2$, and write $u = d(A, K)$, $v = d(B, K)$

$$\frac{1}{2}u + \frac{1}{2}v - \frac{1}{2}|u - v| = \frac{u + v - |u - v|}{2} = \min(u, v),$$

because if $u \leq v$ then $|u - v| = v - u$ and the expression is $(u + v - v + u)/2 = u$.
 Flip the sign of γ and the same algebra gives $\max(u, v)$. $\square$

Why this matters practically

One merge loop, four constants. That is why a single $O(n^2)$ -memory implementation serves every linkage — and why the proximity matrix is the only thing HAC ever has to keep.

In code: the recurrence reproduces SciPy

```
for method in ("single", "complete", "average", "centroid", "ward"):
    h = lance_williams(P6, method) # uses only the recurrence
    print(method, h, linkage(P6, method=method)[: , 2])
```

single	LW	1.0000	2.0000	2.2361	3.1623	4.0000		scipy same		match True
complete	LW	1.0000	2.2361	4.0000	4.4721	8.6023		scipy same		match True
average	LW	1.0000	2.2361	3.2694	4.0000	5.6240		scipy same		match True
centroid	LW	1.0000	2.2361	3.1623	4.0000	5.2202		scipy same		match True
ward	LW	1.0000	2.2361	4.0000	4.4721	8.5245		scipy same		match True

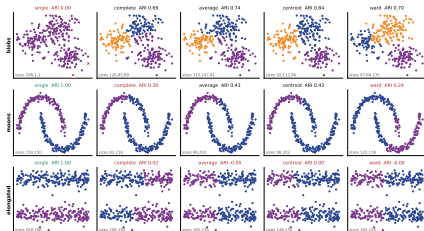
The last average-linkage height, **5.6240**, is the mean of the eight cross distances we computed by hand — reached by the recurrence without ever looking at a coordinate. Centroid's 5.2202 and Ward's 8.5245 are likewise the two numbers from the worked example.

All five on the six points, and their $k = 2$ answers

single	C+D@1.000	B+CD@2.000	A+BCD@2.236	F+ABCD@3.162	E+FABCD@4.000
complete	C+D@1.000	A+B@2.236	E+F@4.000	CD+AB@4.472	EF+CDAB@8.602
average	C+D@1.000	A+B@2.236	CD+AB@3.269	E+F@4.000	CDAB+EF@5.624
centroid	C+D@1.000	A+B@2.236	CD+AB@3.162	E+F@4.000	CDAB+EF@5.220
ward	C+D@1.000	A+B@2.236	E+F@4.000	CD+AB@4.472	EF+CDAB@8.524

Four of the five recover $\{A, B, C, D\}$ and $\{E, F\}$ at $k = 2$.
Only single linkage does not — it gives $\{A, B, C, D, F\}$ and the lone point $\{E\}$.

Linkage is a belief about shape — measured



Single says

connected

anything you can walk across in small steps

Complete says

small diameter

a set in which no two members are far apart

Ward says

low variance

a set that is well described by its mean

Adjusted Rand index against the known truth; red crosses mark clusters of three points or fewer. **Linkage is not a tuning knob** — choose the belief that matches your data.

The numbers behind that grid

data	linkage	ARI	sizes	data	linkage	ARI	sizes
blobs	single	0.000	298,1,1	moons	single	1.000	150,150
blobs	complete	0.692	126,85,89	moons	complete	0.297	82,218
blobs	average	0.741	112,107,81	moons	average	0.425	98,202
blobs	centroid	0.844	92,112,96	moons	centroid	0.425	98,202
blobs	ward	0.697	97,68,135	moons	ward	0.241	122,178
elongated	single	1.000	200,200	elongated	average	-0.002	165,235
elongated	complete	0.017	208,192	elongated	ward	-0.002	165,235

Single linkage is the only one that follows a curve

ARI **1.000** on the moons and on two long bands, where everything else is near-random. It walks the arc one neighbour at a time; the shape is irrelevant to it.

And it collapses on touching blobs

298/1/1, ARI **0.000**. The blobs touch, so the chain runs straight through; the only merges left for the top of the tree are the two most isolated points.

And complete linkage and Ward break long clusters

Look at the bottom row again

The truth is two horizontal bands. Complete linkage and Ward return a **left half and a right half**: ARI 0.017 and -0.002 .

They are not confused

They are doing their job. A long thin band has enormous diameter and enormous variance. A method that minimises either of those genuinely prefers two compact halves.

The mismatch is between the objective and the data

Not inside the algorithm. There is no bug to fix and no parameter to tune. If your clusters are long and thin, do not ask a variance-minimising method to find them.

This is the single most useful sentence in the lecture for the lab.

Predict first #3 — twelve points in the gap

Commit to an answer

Two clean Gaussian clusters, sixty points each, at $x = 0$ and $x = 8$. The nearest point of one to the nearest point of the other is **5.5714** away. Every linkage separates them perfectly.

Now scatter **twelve** noise points in a line across the gap — ten per cent of the data, none of it near either cluster.

What does single linkage give at $k = 2$? What do complete and Ward give?

Predict first #3 — twelve points in the gap

Commit to an answer

Two clean Gaussian clusters, sixty points each, at $x = 0$ and $x = 8$. The nearest point of one to the nearest point of the other is **5.5714** away. Every linkage separates them perfectly.

Now scatter **twelve** noise points in a line across the gap — ten per cent of the data, none of it near either cluster.

What does single linkage give at $k = 2$? What do complete and Ward give?

Single: [**131**, **1**], ARI **0.000**.

Complete and Ward: [70, 62], ARI **1.000**.

Measured: the chaining collapse

bridge	linkage	height at which the two true clusters fuse	k = 2 sizes	ARI
0	single	5.571	[60, 60]	1.000
0	complete	10.381	[60, 60]	1.000
0	ward	62.573	[60, 60]	1.000
4	single	2.005	[62, 62]	1.000
8	single	0.869	[67, 61]	1.000
12	single	0.565	[131, 1]	0.000
12	complete	10.381	[70, 62]	1.000
12	ward	61.146	[70, 62]	1.000
16	single	0.434	[135, 1]	0.000
16	complete	10.381	[70, 66]	1.000

Read the single column downwards

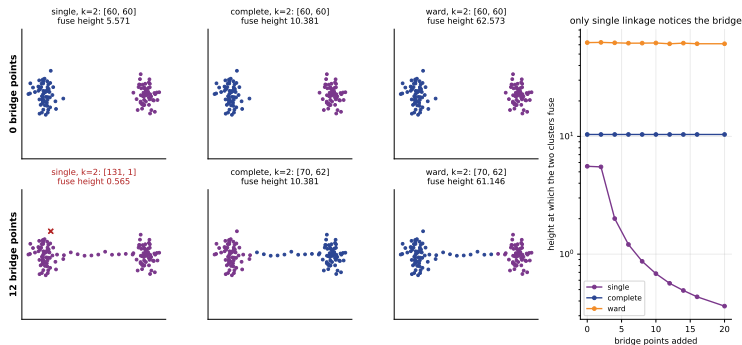
5.571 → 2.005 → 0.869 → 0.565 → 0.434

A factor of ten, from twelve stray points.

The other two do not move

Complete fuses at 10.381 whatever you do — that is $\max_{a,b} d(a,b)$ across the two clusters, and no bridge point can reduce a maximum.

Drawn: the same result



Bottom left: single linkage's "two clusters" are 131 points and the red cross. Right: fusion height against bridge size, log scale — only the purple line moves.

Why: single linkage never pays the gap

nb	spacing	bottleneck step on the left-bridge-right path	single fuse height
0	nan	5.5714	5.5714
4	2.0000	2.0021	2.0048
8	0.8571	0.8687	0.8687
12	0.5455	0.5634	0.5653
16	0.4000	0.4242	0.4341

The fusion height is **exactly the largest single step along the path** left → bridge → right — the bottleneck. Single linkage is measuring *connectivity*, and a thin bridge is a connection.

The examinable version of “chaining”

The linkage takes a **minimum** over cross pairs, so *one* short link is enough to fuse two clusters — and a path of noise supplies one. A maximum, or a mean over $n_A n_B$ pairs, cannot be dragged down by a single pair.

Predict first #4 — can a dendrogram go down?

Commit to an answer

Every merge height so far has been at least as large as the one before. That property — **monotonicity** — is what makes “cut at height h ” meaningful.

Is a later merge *always* at least as high as an earlier one? If not, what is the smallest counterexample?

Predict first #4 — can a dendrogram go down?

Commit to an answer

Every merge height so far has been at least as large as the one before. That property — **monotonicity** — is what makes “cut at height h ” meaningful.

Is a later merge *always* at least as high as an earlier one? If not, what is the smallest counterexample?

Single, complete, average and Ward: **always monotone**.

Centroid: **no** — and the counterexample needs **three points**.

The four monotone methods are *reducible*: if A and B are each at least δ from K , so is $A \cup B$. Centroid linkage is not.

By hand: an inversion on three points

$$P_1 = (0, 0), P_2 = (1, 0), P_3 = (0.48, 0.84)$$

$$d(P_1, P_2) = 1.0000, \quad d(P_1, P_3) = 0.9675, \\ d(P_2, P_3) = 0.9879.$$

Smallest is $d(P_1, P_3)$: **merge at** 0.9675. Their centroid is $c = (0.2400, 0.4200)$, and

$$\|c - P_2\| = \sqrt{0.5776 + 0.1764} = \sqrt{0.7540} = \mathbf{0.8683}.$$

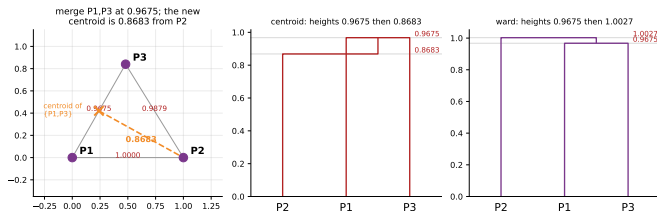
```

scipy centroid heights: 0.9675 0.8683
monotone: False      drop = 0.0991
  single    0.9675 0.9879  monotone True
  complete  0.9675 1.0000  monotone True
  average   0.9675 0.9940  monotone True
  ward      0.9675 1.0027  monotone True
  
```

The geometry, in one sentence

The centroid of a merged pair lies *between* its two members, so it is closer to everything outside than at least one of them was. **Averaging moves you inward.**

Drawn — and it is not a corner case



n=5 uniform points: 6.2%
 n=10 uniform points: 22.2%
 n=25 uniform points: 54.5%
 n=50 uniform points: 75.5%

Fraction of centroid trees
 containing at least one inversion,
 400 random point sets each.

What an inversion costs you

“Cut at height h ” stops being well defined: a cluster formed at 0.97 can sit *underneath* an edge drawn at 0.87. Use average linkage instead — it answers almost the same question and is monotone.

A tree is not a clustering. Cut it.

Two ways

`fcluster(Z, k, "maxclust")` — by count: stop after $n - k$ merges.

`fcluster(Z, t, "distance")` — by height: cut at t , take however many components result.

```

cut at height 4.0 -> 8 clusters
cut at height 6.0 -> 5 clusters,
    20,29,30,45,26
cut at height 8.0 -> 4 clusters,
    49,30,45,26
cut at height 10.0 -> 3 clusters, 49,30,71
cut at height 12.0 -> 3 clusters, 49,30,71
cut at height 16.0 -> 2 clusters, 49,101
  
```

Cutting at 10 and at 12 give the same answer, because no merge happens in between.
That gap is the whole basis of the usual advice about where to cut.

Predict first #5 — how many clusters are in iris?

Commit to an answer

Iris: 150 flowers, four measurements, standardised, Ward linkage. The last eight merge heights are

3.95, 4.06, 4.23, 4.24, 6.61, 8.00, 12.64, 27.25.

The gap heuristic says: cut where the tree has a long stretch with no merges.

What k does the heuristic choose — and iris has three species.

Predict first #5 — how many clusters are in iris?

Commit to an answer

Iris: 150 flowers, four measurements, standardised, Ward linkage. The last eight merge heights are

3.95, 4.06, 4.23, 4.24, 6.61, 8.00, 12.64, 27.25.

The gap heuristic says: cut where the tree has a long stretch with no merges.

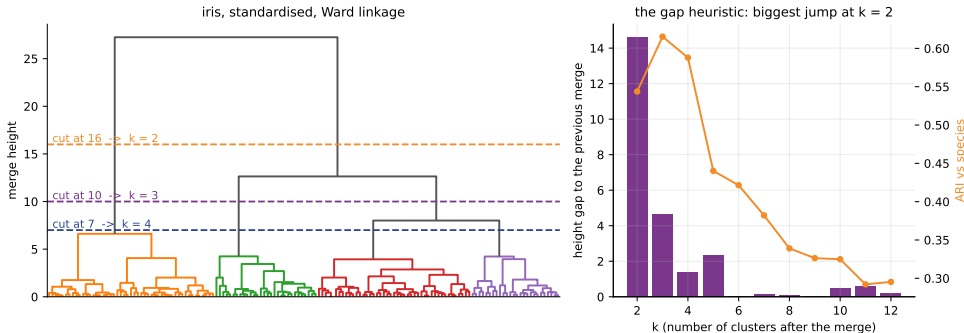
What k does the heuristic choose — and iris has three species.

The biggest gap is $27.25 - 12.64 = \mathbf{14.61}$, the very last merge.

The heuristic says $k = 2$. Silhouette also says $k = 2$ (0.5770 against 0.4467).

ARI against the species is best at $k = 3$ (0.6153 against 0.5438).

Drawn: the tree, the gaps, and the truth



Right-hand panel: the height gap at each k (bars) against ARI with the true species (orange). They peak in different places.

Be honest about what the gap rule is

It is a heuristic, not a criterion

It is not wrong on iris — two species genuinely overlap. But the rule **almost always points at small k** : merge heights grow super-linearly towards the root, so the last gap is nearly always the largest.

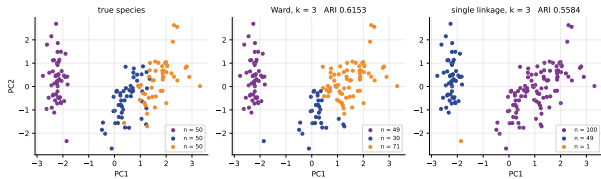
What to do instead

The whole gap profile, not just the maximum. Cross-check with silhouette. Check the cluster sizes — “two clusters” of 149/1 is outlier-peeling, not structure. Then domain knowledge.

The deeper point

Two unsupervised criteria agreed with each other and disagreed with the labels. That is normal. **The number of clusters in the data and the number of classes in the label column are different quantities.**

What the $k = 3$ cut actually did



	cl 1	cl 2	cl 3
species 0	49	1	0
species 1	0	27	23
species 2	0	2	48

Setosa recovered almost exactly. Versicolor and virginica split 27/23 and 2/48 — the boundary is genuinely fuzzy in the measurements.

Right-hand panel: **single linkage on the identical data** gives 100/49/1 — one giant cluster and a stray point. Same failure mode as the touching blobs.

Two decisions that matter more than the linkage

Scaling

raw centimetres	ARI = 0.7312
standardised	ARI = 0.6153
one column x100	ARI = 0.2879

On iris, standardising made it **worse**: the four columns are already in the same unit, and the larger variance of the petal measurements is real signal.

The dissimilarity itself (wine, average linkage)

euclidean	ARI	-0.0054	3,174,1
cityblock	ARI	0.4730	51,126,1
cosine	ARI	0.7847	68,52,58
correlation	ARI	0.8224	56,58,64

Same linkage, same rows. -0.0054 against 0.8224.

Both of these change the answer more than the choice of linkage does. The previous lecture was about exactly this, and **this is where it pays**.

Predict first #6 — can you cluster 100,000 rows?

Commit to a number

You have 100,000 customer records and a laptop with 16 GiB of memory. You want a dendrogram.

How much memory does the proximity matrix alone need?

Predict first #6 — can you cluster 100,000 rows?

Commit to a number

You have 100,000 customer records and a laptop with 16 GiB of memory. You want a dendrogram.

How much memory does the proximity matrix alone need?

$$\frac{100000 \times 99999}{2} \times 8 \text{ bytes} = 4,999,950,000 \times 8 = \mathbf{37.3 \text{ GiB}}$$

And that is the *compact* triangular form, before the algorithm allocates anything else.

This one number is why hierarchical clustering does not appear in production pipelines over large data.

Space: $O(n^2)$, and it is the binding constraint

n	pairs $n(n-1)/2$	condensed float64	full square
100	4,950	38.7 KiB	78.1 KiB
1,000	499,500	3.8 MiB	7.6 MiB
10,000	49,995,000	381.4 MiB	762.9 MiB
50,000	1,249,975,000	9.3 GiB	18.6 GiB
100,000	4,999,950,000	37.3 GiB	74.5 GiB
1,000,000	499,999,500,000	3.6 TiB	7.3 TiB
measured: pdist on 10,000 points returns float64 of shape (49995000,) -> 381.4 MiB			

Turn it around

8 GiB of memory $\Rightarrow n < 46,341$.

64 GiB of memory $\Rightarrow n < 131,072$.

And that assumes nothing else is resident.

No algorithm can rescue this

The memory is in the *input*, not in the merge loop. float32 buys you a factor of $\sqrt{2}$ in n and changes nothing structural.

```
fitted exponent p in t = c * n^p : single 2.18 complete 2.28 ward 2.27  
centroid 2.31 sklearn ward 2.11
```

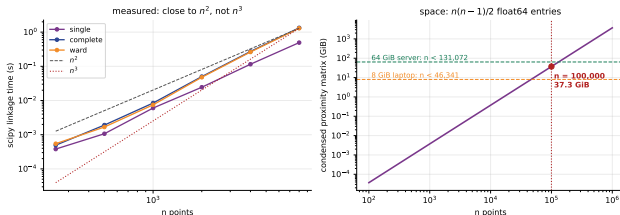
Seconds are machine-dependent — yours will differ.

```
n= 100 naive 0.0124 s scipy 0.00021 s
n= 200 naive 0.0853 s scipy 0.00036 s
n= 400 naive 0.7196 s scipy 0.00074 s
n= 800 naive 6.1881 s scipy 0.00274 s
```

fitted exponent **3.00** — every doubling of n costs **eight** times as much.

Quote the exact count, not the seconds: the loop makes $\binom{n+1}{3}$ comparisons, which is $(n+1)/3 = 267\times$ the size of the matrix at $n = 800$.

Drawn: both curves — and what the alternative costs



k-means, $n = 8,000$	0.051 s	k-means, $n = 50,000$	0.206 s
k-means, $n = 200,000$	0.406 s	(the data itself is 7.6 MiB)	

Hierarchical clustering is a small- n method

Comfortable under a few thousand; unavailable above about 50,000. If you need a hierarchy over large data, **summarise first** — run k -means with a large k , or use BIRCH — then run HAC on the summaries.

Why nobody builds the tree top-down

n = 10	agglomerative first step:	45 pairs	divisive: 511 two-way splits
n = 20	agglomerative first step:	190 pairs	divisive: 524,287 splits
n = 50	agglomerative first step:	1,225 pairs	divisive: 562,949,953,421,311 splits
n = 100	agglomerative first step:	4,950 pairs	divisive: 6.34e+29 splits

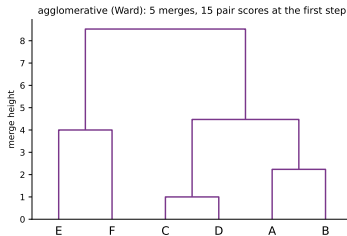
$\binom{n}{2}$ against $2^{n-1} - 1$

At $n = 100$ that is 6.34×10^{29} candidate splits for the *first decision alone*. Exhaustive divisive clustering does not exist beyond toy sizes.

So it is done heuristically

DIANA moves the object with the largest average dissimilarity into a splinter group, repeatedly. **Bisecting k-means** runs *k*-means with $k = 2$ and recurses.

On six points the exhaustive search is affordable



divisive: exhaustive optimal bisection, top down

$\{ABCDEF\} \rightarrow \{ABCD\} \mid \{EF\}$ SSE drop 36.333

$\{ABCD\} \rightarrow \{AB\} \mid \{CD\}$ SSE drop 10.000

$\{EF\} \rightarrow \{E\} \mid \{F\}$ SSE drop 8.000

$\{AB\} \rightarrow \{A\} \mid \{B\}$ SSE drop 2.500

$\{CD\} \rightarrow \{C\} \mid \{D\}$ SSE drop 0.500

first step searches $2^{6-1} - 1 = 31$ splits

The optimal top-down tree is the Ward tree, backwards

And the first split's SSE drop, **36.333**, is exactly the $\Delta(\{A, B, C, D\}, \{E, F\})$ we computed by hand. A happy accident of a small clean example — not guaranteed in general.

```
wine, 178 rows, standardised, k = 3
Ward HAC           ARI 0.7899
bisecting k-means  ARI 0.7062
n=8000  Ward 1.313 s
          bisecting 0.005 s (260x)
```

Comparable quality, two orders of magnitude faster, $O(m)$ memory. The exact version: Ward needs all $61,956,000$ distances first.



So why is agglomerative the default?

It is deterministic

Bisecting k -means depends on a random initialisation; run it twice and you may get two trees. HAC gives the same tree every time.

It gets the bottom of the tree right

Divisive methods make their *coarsest* decisions first, on the least information, and a bad first split can never be repaired.

Care about fine structure — which small groups are most alike? Build **bottom-up**.
Care only about the top few levels of a large data set? Build **top-down**.

Summary — fourteen lines

1. A hierarchical clustering is a **nested family** of partitions, one for every k , drawn as a dendrogram.
2. Leaf order means nothing; **node heights mean everything**.
3. HAC: start from n singletons, merge the two closest clusters $n - 1$ times. Greedy, deterministic, **irrevocable**.
4. Its only input is the **proximity matrix** — so it works on any dissimilarity, except centroid and Ward, which need Euclidean points.
5. Single = min, update by element-wise minimum. Complete = max, update by element-wise maximum; the merge height is the cluster diameter.
6. Same six points, different trees: 1, 2, 2.2361, 3.1623, 4 against 1, 2.2361, 4, 4.4721, 8.6023.
7. Average = mean of the $n_A n_B$ cross distances; centroid = $\|c_A - c_B\|$; Ward = $\sqrt{2\Delta}$ for the within-cluster sum-of-squares increase Δ .
8. **Lance-Williams** unifies all five: one recurrence, four constants.
9. Linkage encodes a belief about shape. Single: ARI 1.000 on moons, ARI 0.000 on touching blobs. Complete and Ward cut long clusters across the middle.
10. **Chaining**: twelve bridge points take single linkage from [60,60] to [131,1]. Complete and Ward do not move.
11. **Centroid linkage inverts** — 0.9675 then 0.8683 on three points — and a non-monotone dendrogram cannot be cut.
12. Cut by count or by height. The largest gap is a **heuristic** and it points at small k : iris says 2, the species say 3.
13. Scaling and the dissimilarity matter more than the linkage: wine, average linkage, ARI -0.0054 Euclidean against 0.8224 correlation.
14. **Cost**: $O(n^2)$ space always. At $n = 100,000$ that is 37.3 GiB, and the discussion is over.

Before the next lecture

Do this with a pen

Take any five or six objects, write the proximity matrix, and run **both** single and complete linkage to completion, rewriting the matrix after every merge. Draw both dendrograms. Then check the heights against `scipy.cluster.hierarchy.linkage`.

This is the examinable calculation. Twenty minutes now saves you the marks later.

Read

The notes: every matrix above is written out, and the practice questions include three more hand runs with full solutions.

Next up

Partitioning and density-based clustering. *k*-means, *k*-medoids, CLARA and DBSCAN — methods that scale to the 200,000 rows this one cannot touch, at the price of having to choose *k* in advance and getting a different answer every run.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.; Ester, Kriegel, Sander and Xu, "A density-based algorithm for discovering clusters (DBSCAN)." KDD 1996.